

RNNs and Backpropagation

How This Model Reads a Sequence and Learns from Loss

The RNN path in this project is:

`image sequence -> RNN loop -> final memory -> digit scores`

- The image is treated like a sequence of input steps.
- The RNN reads one step at a time.
- It keeps a memory called the hidden state.
- After the last step, that memory is used to predict the digit.

Step 1: Input Shape

In `model/model.py`, the model starts with:

```
B, L, C = inputs.shape
```

Name	Meaning
B	batch size: how many images
L	sequence length: how many steps
C	channels: numbers per step

For grayscale MNIST, each pixel step usually has one brightness value, so $C = 1$.

Step 2: Put Sequence First

The input starts as:

```
[batch, length, channels]
```

Then the code changes it:

```
inputs = inputs.transpose(0, 1)
```

Now it is:

```
[length, batch, channels]
```

This makes it easy for the RNN to read step 1, then step 2, then step 3.

Step 3: Start with Empty Memory

Before reading the sequence, the RNN creates the first hidden state:

```
state = self.cell.default_state(inputs[0], max_batch_size)
```

```
state = empty memory
```

At the beginning, the RNN has not seen any pixels yet, so its memory starts as zeros.

Step 4: Loop Through the Sequence

In `model/rnn.py`, the RNN reads one input step at a time:

```
for input in torch.unbind(inputs, dim=0):  
    output, new_state = self.cell.forward(input, state)  
    state = new_state
```

current step + old memory $\rightarrow$ new memory

This is the main RNN idea: the next step depends on what the model already remembers.

Step 5: Update the Memory

The actual memory update happens in `model/rnncell.py`:

```
hidden_preact = self.W_hx(input) + self.W_hh(h)
hidden = self.hidden_activation_fn(hidden_preact)
```

```
return hidden, hidden
```

- `self.W_hx(input)` looks at the current input step.
- `self.W_hh(h)` looks at the previous memory.
- The activation turns the combined value into the new memory.

Step 6: What the Activation Does

The activation is applied after input information and memory information are added.

```
new memory = activation(input part + memory part)
```

- It turns raw numbers into hidden-state values.
- It makes the model nonlinear, so it can learn shapes and patterns.
- It controls how strongly each memory feature turns on or off.

Why tanh?

Vanilla RNNs often use $\tanh$ for the hidden activation.

```
large negative -> near -1  
0 -> 0  
large positive -> near 1
```

- It keeps memory values between -1 and 1.
- It is centered around zero.
- It allows negative, positive, and neutral memory values.

Step 7: Use the Final Memory

After all input steps are read, `model/model.py` keeps the final state:

```
_, state = self.rnn(inputs, init_state=initial_state,  
                    return_output=False)  
state = self.rnn.output(state)  
return self.output_mlp(state)
```

final memory -> output layer -> 10 digit scores

So the RNN summarizes the image first, then the classifier predicts the digit.

From Pixels to Features

The model does not jump directly from pixels to the answer.

784 pixel brightness values -> 256 features -> 10 scores

- The 256 features are learned numbers that summarize the image.
- 256 comes from `hidden_size`: 256.
- If `hidden_size` were 64, the model would use 64 features.

Final Output Layer

In `model/model.py`, the final output layer is built here:

```
backbone_output_size = self.rnn.output_size()
sizes = [backbone_output_size] + output_hiddens + [output_size]
self.output_mlp = nn.Sequential(
    *[nn.Linear(sizes[i], sizes[i+1])
      for i in range(len(sizes)-1)]
)
```

For MNIST with `hidden_size = 256`, this becomes:

```
nn.Linear(256, 10)
```

Linear Layer Meaning

A linear layer combines features with learned weights:

$$\text{scores} = \text{features} \times W + b$$

Part	Shape
features	(256)
W conceptually	(256, 10)
b	(10)
scores	(10)

PyTorch stores the linear weight as (10, 256), but the idea is the same.

Batch, X, Y, and Training Step

Name	Meaning
batch_x	input images as pixel sequences
batch_y	correct digit labels
out	model output scores
loss	how wrong the scores are

```
batch_x, batch_y, *len_batch = batch
out = self.model(batch_x, len_batch)
loss = self.dataset.loss(out, batch_y, len_batch)
return loss
```

Forward Pass and Output Scores

The forward pass means the model makes a prediction using its current weights.

pixels -> model -> 10 digit scores

Suppose the image shows the digit 3:

0	1	2	3	4	5	6	7	8	9
0.1	0.2	-0.5	5.0	0.0	-1.0	0.3	0.4	-0.7	0.2

The highest score is for digit 3, so the model predicts 3.

Weights for Digit 3

Each digit score is a weighted combination of the 256 features.

$$\text{score for 3} = f_1 w_1 + f_2 w_2 + \cdots + f_{256} w_{256} + b$$

- The weights connected to `score_3` are the weights for digit 3.
- At first, these weights are random.
- During training, they become useful for making images of 3 produce high `score_3`.

How Digit 3 Is Learned

Suppose the image is actually a 3:

```
batch_x = image pixels,    batch_y = 3
```

If the model predicts 7, then `score_7` is too high, `score_3` is too low, and the loss becomes high.

```
out = self.model(batch_x, len_batch)
loss = self.dataset.loss(out, batch_y, len_batch)
return loss
```

Backpropagation uses this loss to change the weights a little.

Repeated Learning

One example changes the weights only a little.

pixels -> scores -> loss -> gradients -> update

- After many images of 3, score_3 becomes better for digit 3.
- After many images of other digits, the model learns when not to make score_3 high.
- Learning happens over many batches and epochs.

Loss is one number that says how wrong the model is.

- Low loss means the prediction is good.
- High loss means the prediction is bad.
- For MNIST, this project uses cross entropy loss.

```
class MulticlassClassification(Task):  
    @staticmethod  
    def loss(logits, y, len_batch=None):  
        return F.cross_entropy(logits, y)
```

Chain Rule and Backpropagation

Backpropagation uses the chain rule to calculate gradients.

$$\frac{dL}{dx} = \frac{dL}{dy} \times \frac{dy}{dx}$$

For a model:

weight $\rightarrow$ feature $\rightarrow$ score $\rightarrow$ loss

$$\frac{dLoss}{dWeight} = \frac{dLoss}{dScore} \times \frac{dScore}{dFeature} \times \frac{dFeature}{dWeight}$$

Backward Pass

The backward pass starts from the loss and moves backward through the model.

loss -> scores -> features -> weights

- It uses the chain rule.
- It calculates gradients for the weights.
- This is backpropagation.

Weight Update

$$\text{new weight} = \text{old weight} - \text{learning rate} \times \text{gradient}$$

- Backpropagation calculates gradients.
- The optimizer uses gradients to update weights.
- The new weight can become smaller or larger.
- The goal is smaller loss, not smaller weights.